

FastAPI + Pydantic

ফাউন্ডেশন ও ডেটা ভ্যালিডেশন

BaseModel থেকে শুরু করে Field Validation পর্যন্ত — ডেটাকে পরিষ্কার ও নিরাপদ রাখার সম্পূর্ণ গাইড

Python 3.10+

FastAPI 0.110+

Pydantic v2

বাংলা নোট

সূচিপত্র

1. সহজ ভাষায় পরিচিতি — FastAPI কী এবং Pydantic কেন দরকার ?
2. বিস্তারিত ব্যাখ্যা — BaseModel, Field, Validator
3. Math / Theory — Type System ও Validation Logic
4. Code Example — সম্পূর্ণ ব্যবহারিক উদাহরণ
5. Visual Diagram — Request Flow Architecture
6. Real-world Use Cases
7. Pros & Cons
8. Common Mistakes & Gotchas
9. Related Topics
10. Memory Tricks



বাস্তব জীবনের উদাহরণ — রেস্টোরার অর্ডার সিস্টেম

কল্পনা করো তুমি একটি রেস্টোরার ক্যাশিয়ার। একজন কাস্টমার এসে বলল: "আমাকে ৩টা বিরিয়ানি দাও, দাম -৫০ টাকা এবং আমার ফোন নম্বর হলো 'আবুল'।"

তুমি কি এই অর্ডার নেবে? অবশ্যই না! কারণ — দাম নেগেটিভ হতে পারে না, ফোন নম্বর সংখ্যা হতে হবে। **Pydantic** হলো সেই ক্যাশিয়ার যে অর্ডার নেওয়ার আগে সব যাচাই করে।

FastAPI কী?

FastAPI হলো Python-এর একটি আধুনিক Web Framework যা দিয়ে খুব দ্রুত REST API বানানো যায়। নামেই বোঝা যায় — *Fast*। এটি Python-এর **Type Hint** সিস্টেমকে কাজে লাগিয়ে automatically documentation তৈরি করে এবং data validate করে।

Pydantic কী?

Pydantic হলো একটি Python library যা **Data Validation** এবং **Settings Management**-এর জন্য ব্যবহৃত হয়। FastAPI এর ভেতরে Pydantic ব্যবহার করে। তুমি যখন FastAPI দিয়ে API বানাও, তখন ডেটার structure ও নিয়ম Pydantic দিয়ে সংজ্ঞায়িত করতে হয়।

এটি কোন সমস্যা সমাধান করে?

১

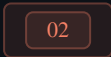
Garbage In, Garbage Out: ব্যবহারকারী ভুল ডেটা পাঠালে Database corrupt হয়ে যায়। Pydantic এটা আগেই আটকায়।

২

Manual Checking ঝামেলা: প্রতিটা field আলাদাভাবে check করা বোরিং ও error-prone। Pydantic এটা automatically করে।



Documentation: Pydantic model থেকে FastAPI automatically Swagger UI বানায়, আলাদা কোনো document লিখতে হয় না।



বিস্তারিত ব্যাখ্যা (**Deep Explanation**)

① Pydantic BaseModel

`BaseModel` হলো Pydantic-এর মূল class। তুমি এটা থেকে inherit করে নিজের data model বানাও। প্রতিটি field-এ Python type hint দিতে হয়।

CONCEPT: SCHEMA = চুক্তি

BaseModel দিয়ে তুমি একটা "চুক্তি" তৈরি করছো: "এই endpoint-এ আসা ডেটা অবশ্যই এই ফরম্যাটে থাকতে হবে।" Pydantic সেই চুক্তি enforce করে।

② Field() — বিস্তারিত নিয়ন্ত্রণ

শুধু type hint দিয়ে সব নিয়ন্ত্রণ করা যায় না। যেমন: বয়স ০-১২০ এর মধ্যে থাকতে হবে, নাম অন্তত ২ অক্ষর হতে হবে। এজন্য `Field()` ব্যবহার করি।

FIELD PARAMETER	কাজ	উদাহরণ
default	ডিফল্ট মান সেট করা	<code>Field(default=18)</code>
gt / ge	greater than / greater than or equal	<code>Field(gt=0)</code>
lt / le	less than / less than or equal	<code>Field(lt=120)</code>

FIELD PARAMETER	কাজ	উদাহরণ
min_length	string-এর সর্বনিম্ন দৈর্ঘ্য	Field(min_length=2)
max_length	string-এর সর্বোচ্চ দৈর্ঘ্য	Field(max_length=50)
pattern	Regex pattern match করা	Field(pattern=r'^\d{11}\$')
description	Swagger UI-তে description দেখায়	Field(description="ব্যবহারকারীর নাম")
alias	JSON-এ ভিন্ন নাম ব্যবহার করা	Field(alias="user_name")

③ Optional Fields

কিছু field থাকতেও পারে, নাও থাকতে পারে। এজন্য `Optional[type]` বা `type | None` ব্যবহার করা হয়।

④ Nested Models — Model-এর ভেতরে Model

বাস্তব ডেটা সবসময় flat (সমতল) হয় না। যেমন: একজন User-এর একটি Address থাকতে পারে, Address-এ City, Street আলাদা। Pydantic-এ এক model-কে অন্য model-এর field হিসেবে ব্যবহার করা যায়।

⑤ @field_validator — Custom Validation

কখনো কখনো built-in validation যথেষ্ট না। নিজের নিয়ম লিখতে হয়। যেমন: ফোন নম্বর অবশ্যই "01" দিয়ে শুরু হতে হবে। এজন্য `@field_validator` decorator ব্যবহার করি।

⑥ model_config — Model-এর আচরণ নিয়ন্ত্রণ

`model_config` দিয়ে Pydantic model-এর global behavior পরিবর্তন করা যায়, যেমন extra fields allow করবো কি না, alias use করবো কি না ইত্যাদি।

03



Math / Theory

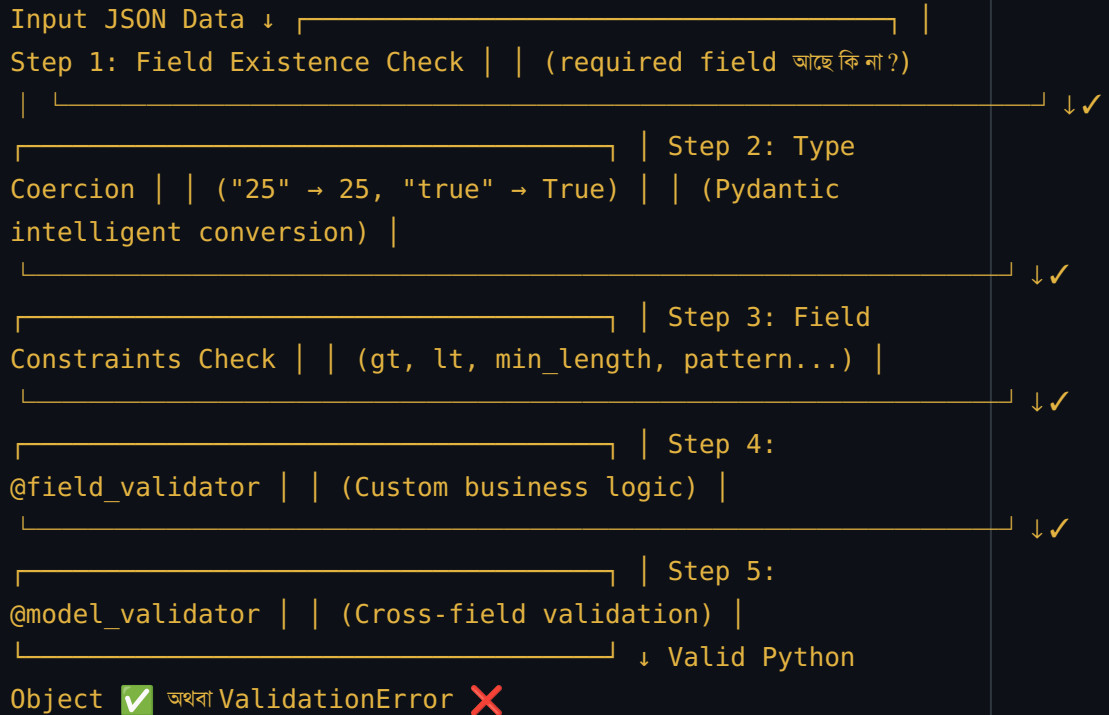
Type System — ধারণা

Python-এর type system হলো একটি annotation system। Python নিজে runtime-এ type enforce করে না (duck typing), কিন্তু Pydantic সেই annotation পড়ে validate করে।

PYTHON TYPE	মানে	উদাহরণ মান	ব্যর্থ উদাহরণ
str	যেকোনো text	"Maruf"	123 (int)
int	পূর্ণ সংখ্যা	25	"পঁচিশ"
float	দশমিক সংখ্যা	3.14	"তিন দশমিক"
bool	সত্য/মিথ্যা	True	"yes" (str)
list[str]	string-এর তালিকা	["a","b"]	[1, 2]
dict[str, int]	string key, int value	{"a": 1}	{"a": "b"}
Optional[str]	str অথবা None	"Maruf" বা None	123

Validation Logic — কিভাবে কাজ করে

Pydantic v2-তে validation-এর ধাপগুলো এরকম:



Type Coercion — স্বয়ংক্রিয় রূপান্তর

Pydantic শুধু strict type match করে না, সে চেষ্টা করে ডেটাকে সঠিক type-এ রূপান্তর করতে:

- JSON থেকে আসা "25" → int field-এ 25 হয়ে যায়
- "true" বা 1 → bool field-এ True হয়
- "3.14" → float field-এ 3.14 হয়

⚠ STRICT MODE

যদি coercion না চাও — মানে শুধু exact type match চাও — তাহলে

`model_config = ConfigDict(strict=True)` ব্যবহার করো।

উদাহরণ ১ — সহজ **BaseModel**

PYTHON — PYDANTIC_BASIC.PY

```
from pydantic import BaseModel
from typing import Optional

# ১. BaseModel থেকে inherit করে নিজের model তৈরি করি
class User(BaseModel):
    name: str          # অবশ্যই string হতে হবে
    age: int            # অবশ্যই integer হতে হবে
    email: str          # অবশ্যই string হতে হবে
    phone: Optional[str] = None # থাকতে পারে, নাও থাকতে পারে

# ২. সঠিক ডেটা দিয়ে object তৈরি
user1 = User(
    name="মারুফ",
    age=25,
    email="maruf@example.com"
    # phone দিইনি — Optional তাই None হবে
)
print(user1)

# ৩. ভুল ডেটা দিলে কী হয়?
try:
    bad_user = User(
        name="আবুল",
        age="পঁচিশ", # ❌ এটা string, int হওয়া উচিত
        email="maruf@test.com"
    )
except Exception as e:
    print(f"Validation Error: {e}")

# ৪. dict থেকে model তৈরি করা
data = {"name": "রাহেলা", "age": 30, "email": "raheela@bd.com"}
```

```
user2 = User.model_validate(data)
print(user2.model_dump()) # dict হিসেবে ফেরত পাওয়া
```

► OUTPUT

```
name='মারুফ' age=25 email='maruf@example.com' phone=None
Validation Error: 1 validation error for User
age
  Input should be a valid integer, unable to parse string as
  an integer [...]
{'name': 'রাহেলা', 'age': 30, 'email': 'rahela@bd.com', 'phone':
None}
```

উদাহরণ ২ — **Field()** দিয়ে **Constraint** যোগ করা



PYTHON — FIELD_CONSTRAINTS.PY

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional

class Product(BaseModel):
    name: str = Field(
        min_length=2,
        max_length=100,
        description="পণ্যের নাম"
    )
    price: float = Field(
        gt=0,          # দাম অবশ্যই শূন্যের বেশি হতে হবে
        le=100000,     # দাম ১ লাখের বেশি হবে না
        description="টাকায় দাম"
    )
    quantity: int = Field(
        ge=0,          # পরিমাণ শূন্য বা তার বেশি হতে হবে
        default=0       # ডিফল্ট ০
    )
    category: str = Field(default="সাধারণ")

# সঠিক পণ্য
biryani = Product(name="চিকেন বিরিয়ানি", price=250.0, quantity=10)
```



```

print(biryani)

# ভুল — নেগেটিভ দাম
try:
    bad = Product(name="ভাত", price=-50)
except Exception as e:
    print(f"❌ Error: {e}")

```

► OUTPUT

```

name='চিকেন বিরিয়ানি' price=250.0 quantity=10 category='সাধারণ'
❌ Error: 1 validation error for Product
price
  Input should be greater than 0 [type=greater_than, ...]

```

উদাহরণ ৩ — @field_validator দিয়ে Custom Validation



PYTHON — CUSTOM_VALIDATOR.PY

```

from pydantic import BaseModel, Field, field_validator

class BangladeshiUser(BaseModel):
    name: str = Field(min_length=2)
    phone: str = Field(description="বাংলাদেশি ফোন নম্বর")
    nid: str = Field(description="জাতীয় পরিচয়পত্র নম্বর")

# phone field-এর custom validator
@field_validator("phone")
@classmethod
def validate_phone(cls, v: str) -> str:
    # ফোন নম্বর থেকে স্পেস সরিয়ে দাও
    v = v.replace(" ", "")

    # বাংলাদেশি নম্বর "01" দিয়ে শুরু হয় এবং ১১ ডিজিটের
    if not v.startswith("01"):
        raise ValueError("ফোন নম্বর অবশ্যই 01 দিয়ে শুরু হতে হবে")
    if len(v) != 11:
        raise ValueError("ফোন নম্বর অবশ্যই ১১ ডিজিটের হতে হবে")
    if not v.isdigit():

```

```

        raise ValueError("ফোন নম্বরে শুধু সংখ্যা থাকতে হবে")
    return v

# NID validator — ১০ বা ১৭ ডিজিট হতে হবে
@field_validator("nid")
@classmethod
def validate_nid(cls, v: str) -> str:
    v = v.strip()
    if len(v) not in [10, 17]:
        raise ValueError("NID অবশ্যই ১০ বা ১৭ ডিজিটের হতে হবে")
    if not v.isdigit():
        raise ValueError("NID-তে শুধু সংখ্যা থাকতে হবে")
    return v

# সঠিক ডেটা
user = BangladeshiUser(
    name="মারুফ হোসেন",
    phone="01712345678",
    nid="1234567890"
)
print("✅ User created:", user)

# ভুল ফোন নম্বর
try:
    bad = BangladeshiUser(name="করিম", phone="02312345", nid="1234567890")
except Exception as e:
    print("❌ Error:", e)

```

► OUTPUT

```

✅ User created: name='মারুফ হোসেন' phone='01712345678'
nid='1234567890'
❌ Error: 1 validation error for BangladeshiUser
phone
  Value error, ফোন নম্বর অবশ্যই 01 দিয়ে শুরু হতে হবে [...]

```

উদাহরণ ৪ — FastAPI-তে Pydantic ব্যবহার



PYTHON — MAIN.PY (FASTAPI)

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from datetime import datetime

app = FastAPI(title="স্মার্ট কুকিং সিস্টেম API")

# — Request Schema (ব্যবহারকারী যা পাঠাবে) —
class CookingRequest(BaseModel):
    dish_name: str = Field(
        min_length=2,
        max_length=100,
        description="রান্নার নাম (বাংলা বা ইংরেজি)"
    )
    servings: int = Field(
        ge=1, le=10,
        default=2,
        description="কতজনের জন্য রান্না করতে হবে"
    )
    spice_level: str = Field(
        default="medium",
        description="ঝাল স্তর: low / medium / high"
    )

    @field_validator("spice_level")
    @classmethod
    def check_spice(cls, v):
        allowed = ["low", "medium", "high"]
        if v not in allowed:
            raise ValueError(f"ঝাল স্তর অবশ্যই {allowed} থেকে একটি হতে হবে")
        return v

# — Response Schema (API যা ফেরত দেবে) —
class CookingResponse(BaseModel):
    order_id: str
    dish_name: str
    estimated_time_min: int
    status: str
    created_at: str

# — API Endpoint —

```

```
@app.post("/cook", response_model=CookingResponse)
async def start_cooking(request: CookingRequest):
    # FastAPI automatically validate করবে CookingRequest দেখে
    # ভুল ডেটা আসলে automatic 422 error return করবে

    return CookingResponse(
        order_id="ORD-001",
        dish_name=request.dish_name,
        estimated_time_min=30 * request.servings,
        status="শুরু হয়েছে",
        created_at=str(datetime.now())
    )
```

✓ এই CODE চালানোর নিয়ম

`pip install fastapi uvicorn pydantic` চালাও, তারপর `uvicorn`
`main:app --reload` দিলে server উঠবে। `http://localhost:8000/`
`docs` খুললে Swagger UI দেখবে।

উদাহরণ ৫ — Nested Model (ভেতরে Model)



PYTHON — NESTED_MODELS.PY

```
from pydantic import BaseModel, Field
from typing import List, Optional

# ছোট model — ঠিকানা
class Address(BaseModel):
    street: str
    city: str = Field(default="রাজশাহী")
    postal_code: str

# ছোট model — পণ্য
class OrderItem(BaseModel):
    product_name: str
    quantity: int = Field(ge=1)
    unit_price: float = Field(gt=0)
```

```
# বড় model - অর্ডার (ভেতরে দুটো model ব্যবহার করছে)

class Order(BaseModel):
    customer_name: str
    delivery_address: Address # nested model
    items: List[OrderItem] # list of nested model
    note: Optional[str] = None

# সম্পূর্ণ nested structure তৈরি করা
order = Order(
    customer_name="মারুফ হোসেন",
    delivery_address=Address(
        street="পদ্মা আবাসিক, ১৫ নম্বর রোড",
        postal_code="6200"
    ),
    items=[
        OrderItem(product_name="চিকেন বিরিয়ানি", quantity=2, unit_price=250),
        OrderItem(product_name="কোল্ড ড্রিংক", quantity=2, unit_price=40.0)
    ]
)

print(order.model_dump_json(indent=2)) # সুন্দর JSON হিসেবে দেখাও
```

► OUTPUT

```
{
  "customer_name": "মারুফ হোসেন",
  "delivery_address": {
    "street": "পদ্মা আবাসিক, ১৫ নম্বর রোড",
    "city": "রাজশাহী",
    "postal_code": "6200"
  },
  "items": [
    {"product_name": "চিকেন বিরিয়ানি", "quantity": 2, "unit_price": 250.0},
    {"product_name": "কোল্ড ড্রিংক", "quantity": 2, "unit_price": 40.0}
  ],
  "note": null
}
```



FastAPI Request-Response Flow

```

ব্রাউজার / Client | | POST /cook | {"dish_name": "বিরিয়ানি",
"servings": 2} ▼

┌
FastAPI Framework | | | | ① Route Matching | |
@app.post("/cook") → start_cooking() | | | | ② Pydantic
Validation (CookingRequest) | |
├ | | | | dish_name:
"বিরিয়ানি" → ✓ str | | | | servings: 2 → ✓ int (1≤x≤10) | | | |
spice_level: "medium" → ✓ valid | | |
└

| | ✗ যদি ভুল → 422 Unprocessable Entity | | | | ③ Business
Logic | | start_cooking(request) function চলে | | | | ④
Response Validation (CookingResponse) | | return data →
Pydantic serialize করে |
└

200 OK | {"order_id": "ORD-001", "status": "শুরু হয়েছে"} ▼ ব্রাউজার /
Client
  
```

Pydantic Model Hierarchy

```

BaseModel (Pydantic) | └─ User (তোমার Model) | └─ name: str
| └─ age: int = Field(gt=0, lt=120) | └─ email: str | └─
Product (তোমার Model) | └─ name: str = Field(min_length=2) |
└─ price: float = Field(gt=0) | └─ category: str =
"general" | └─ Order (তোমার Model) └─ customer: User ←
Nested Model └─ items: List[Product] ← List of Nested └─
address: Address ← Nested Model
  
```

Request Schema vs Response Schema Pattern

```
UserCreateRequest | | UserResponse |
| | id: int | | email: str | —> | name: str | | password:
str | | email: str | | age: int | | created_at: datetime |
| |
| | (ব্যবহারকারী পাঠায়) (API ফেরত
দেয়) ↑↑ password এখানে password এখানে নেই!(sensitive)(security best
practice)
```

06

✓

Real-world Use Cases

USE CASE	কোথায় PYDANTIC ব্যবহার হয়	উদাহরণ কোম্পানি/প্রজেক্ট
ই-কমার্স অর্ডার সিস্টেম	পণ্যের দাম, পরিমাণ, শিপিং ঠিকানা validate করা	Shopify, Daraz Bangladesh
AI/ML API	Model input (prompt, temperature, max_tokens) validate করা	OpenAI API, Anthropic Claude API
ব্যাংকিং সিস্টেম	ট্রান্সফার পরিমাণ, account number format validate করা	bKash, Nagad
স্বাস্থ্য সেবা		Telemedicine apps

USE CASE

কোথায় **PYDANTIC** ব্যবহার হয়

উদাহরণ কোম্পানি/প্রজেক্ট

রোগীর বয়স, ওষুধের মাত্রা validate করা

তোমার **Smart Cooking System**

রান্নার নাম, servings, উপাদানের পরিমাণ validate করা

তোমার প্রজেক্ট! 🔍

07



Pros & Cons

সুবিধা ✓

অসুবিধা ✗

✓ Automatic validation —
আলাদা code লিখতে হয় না

✗ Learning curve — Pydantic
v1 vs v2 syntax আলাদা

✓ Clear error messages — ঠিক
কোন field-এ কী সমস্যা জানা যায়

✗ Performance overhead —
complex model-এ slightly slow

✓ Auto Swagger
documentation generate হয়

✗ Pydantic v2-তে v1-এর অনেক
syntax কাজ করে না

✓ Type safety — IDE-তে
autocomplete পাওয়া যায়

✗ Circular model reference
handle করা ঝামেলা

✓ JSON serialization/
deserialization automatic

✗ অনেক বড় nested model বুঝতে সময়
লাগে

✓ Python type hints-এর সাথে
seamless integration

✗ Very complex custom
validators লিখতে verbose হতে পারে



Common Mistakes & Gotchas

❌ ভুল ১ — PYDANTIC V1 SYNTAX PYDANTIC V2-তে ব্যবহার করা

অনেক পুরনো tutorial Pydantic v1 দিয়ে লেখা। v2-তে `@validator` কাজ করে না, `@field_validator` ব্যবহার করতে হবে। `dict()` এর বদলে `model_dump()` ব্যবহার করো।

❌ ভুল ২ — MUTABLE DEFAULT VALUE

কখনো `tags: list = []` লিখবে না! সব instance একই list share করবে।
সঠিক উপায়: `tags: list = Field(default_factory=list)`

⚠️ ভুল ৩ — @CLASSMETHOD ভুলে যাওয়া

Pydantic v2-তে `@field_validator` এর সাথে `@classmethod` দেওয়া mandatory। না দিলে warning আসবে এবং ভবিষ্যতে error হবে।

⚠️ ভুল ৪ — REQUEST ও RESPONSE MODEL আলাদা না করা

একই model দিয়ে input ও output handle করলে security risk থাকে।
যেমন: password বা internal_id response-এ চলে যেতে পারে। সবসময় আলাদা `CreateRequest`, `UpdateRequest`, `Response` model বানাও।

💡 ভুল ৫ — NONE CHECK না করা

`Optional[str]` field থেকে সরাসরি `.upper()` call করলে `AttributeError` আসবে। সবসময় `if value is not None` check করো।



আগে যা জানা দরকার ছিল:

[Python Basics](#)[Python Type Hints](#)[Class & Inheritance](#)[JSON এর ধারণা](#)[HTTP Methods \(GET/POST/PUT\)](#)[Decorator \(@\)](#)

পরে যা শেখা উচিত:

[FastAPI Routing & Path Parameters](#)[SQLAlchemy + Pydantic Integration](#)[FastAPI Dependency Injection](#)[JWT Authentication](#)[FastAPI Background Tasks](#)[Alembic Migrations](#)[Docker + FastAPI Deployment](#)

তোমার প্রজেক্টের সাথে সংযোগ:

SMART COOKING SYSTEM CONNECTION

তোমার Cooking System-এ এই Pydantic models কাজে লাগবে:

- `CookingRequest` — dish_name, servings, spice_level validate করবে
- `IngredientSchema` — ingredient name, quantity, unit validate করবে
- `RecipeResponse` — LLM থেকে পাওয়া recipe structure করবে
- `ESP32Command` — edge device-এ পাঠানো command validate করবে



"Pydantic = ব্যাংকের গেটকিপার"

ব্যাংকে ঢোকার আগে যেমন ID card check হয়, API-তে ডেটা ঢোকার আগে Pydantic সেটা check করে। ভুল ID = ঢোকা বন্ধ। ভুল ডেটা = 422 Error।

সহজ মনে রাখার সূত্র — "BFVC"

অক্ষর	মানে	উদাহরণ
B	BaseModel থেকে inherit করো	<code>class User(BaseModel)</code>
F	Field() দিয়ে constraint দাও	<code>age: int = Field(gt=0)</code>
V	Validator দিয়ে custom rule লেখো	<code>@field_validator("phone")</code>
C	Config দিয়ে behavior control করো	<code>model_config = ConfigDict(...)</code>



এক লাইনে সারসংক্ষেপ

"FastAPI হলো দরজা, Pydantic হলো দরজার তালা — ভুল key দিলে ভেতরে ঢোকা যাবে না।"

প্রজেক্টে প্রথম কাজ:

1. `pip install fastapi uvicorn pydantic` ইন্সটল করো

2. `schemas.py` ফাইল বানাও এবং সব Pydantic model রাখো
3. `main.py` -তে FastAPI app তৈরি করো এবং `schemas import` করো
4. `uvicorn main:app --reload` দিয়ে server চালাও
5. `http://localhost:8000/docs` খুলে Swagger UI দেখো